

ModTGCN: Modularity-aware Graph Neural Networks for Text Classification

Rajarshi Misra¹, Aditya Sharma¹, and Vinti Agarwal^{1*} Hari Om Aggrawal²

¹ BITS Pilani, India f20201822p@alumni.bits-pilani.ac.in,
{p20200470,vinti.agarwal}@pilani.bits-pilani.ac.in

² Independent Researcher hariom85@gmail.com

Abstract. Graph-based text classification models typically rely on local neighborhood aggregation and overlook global community structure, despite semantic document graphs exhibiting strong class-consistent clustering. Ignoring this can blur class boundaries and lead to over-smoothing. We propose ModTGCN, a modularity-aware graph neural network for text classification that jointly optimizes cross-entropy and a modularity-based auxiliary objective to promote class-coherent document communities while preserving discriminative representations. The modularity term is computed on a document–document similarity graph derived from transformer embeddings (pretrained or fine-tuned). To improve scalability, we decouple the original heterogeneous TextGCN graph into separate document–word and word–word components, achieving 2×–10× faster training. We further study graph construction strategies, label-aware edge reweighting, and supervision choices for modularity optimization. Experiments on five benchmarks show consistent gains, with larger improvements on complex, low homophily datasets such as Ohsumed and 20NG.

Keywords: Graph Neural Networks · Modularity · Text Classification.

1 Introduction

Text classification remains a fundamental task in natural language processing. Recent advances in transformer-based models such as BERT and large language models (LLMs) have achieved strong zero- and few-shot performance [16,23]. However, these models often require costly full fine-tuning, parameter-efficient adaptation (e.g., adapters, LoRA), or prompt calibration to perform well in supervised settings. An alternative line of work formulates text classification as learning over graph structures, where words and documents are represented as nodes, and edges encode lexical or semantic relationships. Graph neural network (GNN) approaches such as TextGCN[24], TensorGCN[10], BertGCN[9], and VGCBert[11] explicitly model interactions among words and documents, enabling semi-supervised learning through relational propagation. Despite their

* Corresponding author: vinti.agarwal@pilani.bits-pilani.ac.in

effectiveness, most graph-based text classifiers rely primarily on local neighborhood aggregation. Yet semantic document graphs often exhibit clear mesoscopic structure: documents sharing the same label tend to form assortative clusters with dense intra-class connectivity and sparse inter-class links. When this global community structure is ignored, two issues commonly arise: (i) hub-driven shortcuts from high-frequency terms or noisy similarities blur class boundaries, and (ii) over-smoothing in deeper GNNs homogenizes representations across weak community separations, reducing discriminability. Incorporating global structural information during training may therefore improve robustness and class separation [3].

Modularity optimization—originally developed for community detection [13,14]—provides a principled mechanism for modeling such global structure. Motivated by this observation, we introduce **ModTGCN**, a modularity-aware GNN for text classification. Our approach augments standard cross-entropy supervision with a modularity-based auxiliary objective computed on a document–document similarity graph derived from transformer embeddings. The central hypothesis is that explicitly optimizing modular structure aligns learned representations with class-level graph communities, thereby providing global regularization beyond local message passing. Although modularity-aware GNNs have shown promise in other domains [12,22], their use in semi-supervised document classification remains underexplored.

A key challenge in semi-supervised graph learning is propagating supervision from limited labeled nodes while respecting global structure. We address this by computing modularity on a document–document graph constructed from pretrained or fine-tuned SBERT embeddings [20], using gold labels for labeled nodes and TextGCN [24] predictions for unlabeled nodes. This hybrid supervision scheme yields label-efficient and interpretable improvements without requiring expensive LLM fine-tuning, while remaining encoder-agnostic and compatible with future embedding advances.

To further improve scalability, we decouple the original single heterogeneous TextGCN graph into separate document–word (TF–IDF) and word–word (PMI) components. This preserves the original propagation mechanism while substantially reducing computational overhead on large datasets. A detailed complexity analysis is provided in Section 5.3.

To this end, we summarize our contributions as:

- **Modularity-aware GNN with hybrid supervision.** We introduce a joint objective $L = \text{CE} + \lambda(-Q)$ that promotes class-consistent communities on a document–document graph built from language model embeddings, using pseudo-labels for unlabeled nodes.
- **Architectural decoupling of TextGCN.** We reformulate the heterogeneous graph into separate document–word and word–word components, improving scalability without altering the effective decision function.
- **Document–document adjacency strategies.** We compare TF–IDF, cosine, and Gaussian similarity graphs, analyzing accuracy–scalability trade-offs.

- **Empirical validation.** Experiments on five benchmark datasets demonstrate that jointly optimizing classification and modularity consistently improves performance, particularly on structurally complex datasets.

2 Related Work

Existing graph-based text classification methods can be categorized into **GNN-only** and **hybrid GNN-language model (LM)** approaches. **GNN-only** methods such as TextGCN[24] construct heterogeneous document-word graphs using TF-IDF and PMI edges and apply a two-layer GCN for semi-supervised text classification. Subsequent variants extend this design by incorporating multi-view or tensor-based word graphs[10], heterogeneous attention mechanisms[18], or additional lexical nodes (e.g., character and n-grams) to enrich structural information[8]. While effective, these methods rely on large, fine-grained graphs, which incur high computational costs and limit scalability. **Hybrid GNN-LM** models integrate contextual embeddings from pretrained transformers into graph learning. Approaches such as BERTGCN[9] and VGCN-BERT[11] jointly train or fuse language models with graph encoders to combine local contextual signals and global relational structure. Although they improve performance, co-training LMs with GNNs incurs substantial computational overhead.

Modularity in GNNs. Modularity [15] measures community quality by comparing observed within-community edges against a degree-preserving null model. A known limitation is the *resolution limit* [2], which causes small but coherent communities to be merged into larger ones. To alleviate this, resolution-adjusted variants introduce a tunable parameter [19] (with $\gamma > 1$ revealing smaller groups) or adopt *modularity density* [4] and its generalization Q_g [5], which weights communities by internal link density to better retain cohesive clusters. Several works integrate modularity into neural objectives: Modularity regularizers have been added to GAE/VGAE for unsupervised community detection [21], VGAER [17]; Murata & Afzal [12] directly optimize modularity during GNN training, producing embeddings aligned with community structure for clustering tasks. These studies show that modularity-aware learning can uncover latent structure and improve downstream quality. **Connection to our setting:** Prior modularity-aware approaches primarily target unsupervised clustering or community detection. In contrast, we integrate modularity into semi-supervised text classification, aligning graph communities with class labels through hybrid supervision on a document-document similarity graph. This provides a global structural prior that complements local message passing while maintaining computational efficiency via our decoupled graph construction.

3 Proposed Method

3.1 Problem Formulation

Given a corpus of documents $\mathcal{D} = \{d_1, d_2, \dots, d_n\}$ and a corresponding label set $\mathcal{Y} = \{y_1, y_2, \dots, y_n\}$ spanning $\mathcal{C}$ classes, the primary goal is to perform document

classification. During training, document set $\mathcal{D}$ is partitioned into labeled U and unlabeled V sets, and the objective is to learn a mapping function $f : \mathcal{D} \rightarrow \mathcal{Y}$ that can accurately predict the class y_i for each unlabeled documents using graph-based relational structure.

3.2 ModTGCN: Graph Construction

ModTGCN operates over three graphs: (1) a *document-word graph* $\mathcal{G}_d = (\mathcal{V}_d, \mathcal{E}_d)$ constructed via TF-IDF weights, (2) a *word-word graph* $\mathcal{G}_w = (\mathcal{V}_w, \mathcal{E}_w)$, built from PMI-based word co-occurrence, and (3) *document-document similarity graph* for modularity optimization, $\mathcal{G}_{doc} = (\mathcal{V}_{doc}, \mathcal{E}_{doc})$, where edge weights represent node similarity computed using Gaussian (RBF) kernel $S_{ij} = \exp\left(-\frac{\|e_i - e_j\|^2}{2\sigma^2}\right)$ [25], e_i is the embedding of document $i \in \mathcal{V}_{doc}$ from transformers and σ controls the neighborhood sensitivity in kernel space. The first two graphs preserve TextGCN’s propagation structure, while the third introduces global structural supervision.

3.3 Modularity as an Objective Function

Standard GCNs[7] aggregate over L-hop local neighborhoods, modularity introduces global, degree-aware coupling between all node pairs via the null-model term. However, semantic document graphs often exhibit mesoscopic community structure aligned with class labels. To explicitly encourage such structure, we introduce modularity as an auxiliary objective. Modularity, Q measures the deviation of observed intra-community connectivity from a degree-preserving null model. For the label (or community) matrix $P \in \mathbb{R}^{n \times C}$, the modularity is:

$$Q(P) = \frac{1}{2m} \text{Tr}(P^\top B P), \quad B = A - \gamma \frac{k k^\top}{2m}, \quad (1)$$

where A is the adjacency, k the degree vector, m the total edge weight, and γ the resolution parameter. Modularity matrix B quantifies how much the graph can deviate from the null model. Building on this, the modularity loss is:

$$\mathcal{L}_{mod}(P) = -Q(P), \quad (2)$$

which is minimized when the predicted communities (induced by P) have intra-community edge count exceeding the null-model expectation. Importantly, even non-adjacent nodes ($A_{ij} = 0$) contribute via the null-model term $\gamma \frac{k k^\top}{2m}$, giving modularity its global coupling effect.

Modularity gradient under hybrid supervision. We partition the label matrix P as (P_U, P_V) and the modularity matrix B into blocks $\{B_{UU}, B_{UV}, B_{VU}, B_{VV}\}$ based on labeled, U and unlabeled V nodes.

Modularity Q , is computed on $\mathcal{G}_{doc}$, using gold labels for P_U and TextGCN pseudo labels for P_V . We also evaluate a variant that uses soft labels for both P_U and P_V ; see ablation study in Section 6. The gradient with respect to P_V is:

$$\nabla_{P_V} \mathcal{L} = -\frac{1}{m} (B_{VV} P_V + B_{VU} P_U). \quad (3)$$

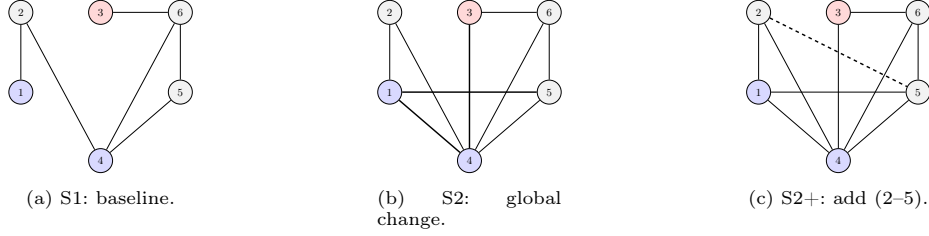


Fig. 1: Toy graphs with labeled nodes $U = \{1, 3, 4\}$ (blue, red, blue) and unlabeled nodes (grey) $V = \{2, 5, 6\}$. Node 2 always connects to nodes 1 and 4 (both A). Blue/red = class A/B. Thick edges are added in S2; dashed in S2+.

The term $B_{VU}P_U$ acts as a degree-corrected supervision field induced by labeled nodes, while $B_{VV}P_V$ couples unlabeled nodes to one another. Expanding the supervision field for node $i \in V$ and class c :

$$[B_{VU}P_U]_{i,c} = \sum_{j \in U} A_{ij} \mathbf{1}[g_j = c] - \frac{k_i}{2m} \sum_{j \in U} k_j \mathbf{1}[g_j = c]. \quad (4)$$

Thus, nodes are encouraged toward classes where observed connectivity exceeds null-model expectation, mitigating hub bias and discouraging degenerate single-cluster assignments.

Global coupling in modularity Q (toy). Figure 1 shows three cases highlighting the global nature of modularity. In *S1 (baseline)*, node 2 links to moderate-degree blue nodes. Because these observed edges exceed the degree-corrected null-model expectation, the supervision field favors the blue class and penalizes red. In *S2 (global change)*, adding (1, 5) and (3, 4) increases blue node degrees and m without altering node 2’s neighborhood, raising the null-model baseline and weakening the blue field, despite its immediate connections remaining unchanged. This demonstrates the dependence of modularity on the *global* degree distribution, rather than local adjacency. In *S2+*, adding (2, 5) changes k_2 and m and activates $B_{VV}P_V$ in (3), so node 2 is not only influenced by labeled neighbors but also by unlabeled neighbors’ soft labels (e.g., a red-leaning node 5 reduces its preference for blue), illustrating unlabeled–unlabeled coupling.

Observation. This mechanism provides (i) *global supervision propagation* via the degree-corrected field $B_{VU}P_U$, allowing few labels to influence distant nodes, and (ii) *unlabeled–unlabeled coherence* via $B_{VV}P_V$, aligning predictions among unlabeled neighbors. Overall, modularity propagates supervision through a degree-aware global field: labeled nodes influence distant regions via the null model, and unlabeled nodes align through mutual coupling. This mechanism complements local GCN aggregation by enforcing mesoscopic community consistency while mitigating hub-driven shortcuts and degenerate single-cluster solutions.

Joint Optimization Objective For decoupled-TextGCN, we use the standard cross-entropy loss on labeled nodes:

$$\mathcal{L}_{CE} = -\text{Tr}\left(Y_U^\top \log \hat{Y}_U\right), \quad (5)$$

where $\text{Tr}(\cdot)$ is the trace, Y_U is the one-hot label matrix, and $\hat{Y}_U$ denotes the predicted label distributions. Predictions are computed as

$$\hat{Y} = \text{softmax}\left(\mathcal{A}_{dw} \left(\mathcal{A}_{dw}^\top W_1 + \mathcal{A}_{ww} W_2\right) W_3\right), \quad (6)$$

with $\mathcal{A}_{dw}$ and $\mathcal{A}_{ww}$ the document-word and word-word adjacencies, and W_1, W_2, W_3 trainable weights. This preserves the original TextGCN propagation while improving scalability (see Section 5.3). The final objective combines classification and modularity:

$$\mathcal{L}_{total} = \lambda \mathcal{L}_{CE} - (1 - \lambda) \mathcal{L}_{mod}, \quad (7)$$

where $\lambda \in [0, 1]$ controls the trade-off between predictive accuracy and community-structure preservation.

4 Experimental setup

4.1 Baselines

We evaluate our method on five benchmark datasets: MR, R8, R52, 20 News-groups (20NG), and Ohsumed as used in TextGCN [24]. A detailed statistics of datasets are available on our website [1]. We compare our framework against three broad categories of models. **GNN-based models:** TextGCN [24], TensorGCN [10], WCTextGCN [8]; **Classification on BERT embeddings:** Logistic Regression(LR) and Linear Probing (a single-layer MLP); **LLM in zero and few-shot settings:** OpenAI-GPT-3, ChatGPT; **Ours:** ModTGCN with $\mathcal{G}_{doc}$ using pre-trained/fine-tuned embeddings.

4.2 Implementation and Evaluation Metrics

We follow TextGCN[24] configuration, including preprocessing, sliding window size (20), dataset splits, and hidden layer size (200). The model is trained with Adam optimizer for up to 300 epochs with early stopping (patience 30). For modularity optimization, hyperparameters including dropout, σ , learning rate η , resolution γ , loss weight λ , and edge-weight parameters (α, β, k) with tuned with Optuna. Performance is reported using micro-F1, averaged over five random seeds for stability. The code is available on GitHub <https://github.com/Rajarshi-Misra/ModTGCN>.

Table 1: Micro-F1 (mean% $\pm$ std) of competing methods on MR, R8, R52, Ohsumed and 20NG. Abbreviations: (P)/(F) — pre-trained/fine-tuned SBERT-**all-mpnet-base** embeddings used to build the doc-doc graph for the modularity term; LR — Logistic Regression. Best results per dataset are shown in **bold**. The homophily values corresponding to each dataset are reported in parentheses.

Model	MR(0.70)	R8(0.50)	R52(0.38)	Ohsumed(0.16)	20NG(0.19)
TextGCN	76.74 $\pm$ 0.20	97.07 $\pm$ 0.13	93.56 $\pm$ 0.18	68.36 $\pm$ 0.56	86.34 $\pm$ 0.09
TensorGCN	77.91 $\pm$ 0.07	98.04 $\pm$ 0.08	95.05 $\pm$ 0.11	70.11 $\pm$ 0.24	87.74 $\pm$ 0.05
WCTextGCN [8]	77.85 $\pm$ 0.34	97.49 $\pm$ 0.20	93.88 $\pm$ 0.34	68.52 $\pm$ 0.20	87.21 $\pm$ 0.54
U-TextGCN [6]	76.41	96.92	93.45	68.24	86.07
BertGCN [9] (Co-trained BERT with TextGCN)	86.0	98.1	96.6	72.8	89.5
Pre-trained embeddings(P)					
LR(P)	83.73	98.12	95.63	71.30	79.06
Linear Probing(P)	83.49 $\pm$ 0.09	98.25 $\pm$ 0.07	95.26 $\pm$ 0.05	69.87 $\pm$ 0.09	79.06 $\pm$ 0.14
ModTGCN(P)	81.45 $\pm$ 0.05	97.55 $\pm$ 0.06	94.54 $\pm$ 0.11	71.97 $\pm$ 0.15	90.6 $\pm$ 0.1
Fine-tuned embeddings(F)					
LR(F)	87.87	98.04	96.73	74.65	87.17
Linear Probing(F)	87.37 $\pm$ 0.20	98.10 $\pm$ 0.12	96.71 $\pm$ 0.11	74.76 $\pm$ 0.17	86.90 $\pm$ 0.10
ModTGCN(F)	88.07$\pm$0.06	98.70$\pm$0.09	96.16 $\pm$ 0.16	77.52$\pm$0.25	91.14 $\pm$ 0.12
ModTGCN(F) warmup	88.04	98.40	96.14	76.34	89.91

5 Results

5.1 Baseline Performance

Table 1 shows that modularity-based optimization substantially improves performance over TextGCN and related GNN baselines. Using pre-trained embeddings, **ModTGCN(P)** achieves clear gains over TextGCN across datasets, with particularly strong improvements on MR(+4.7), Ohsumed(+3.6) and 20NG(+4.3). To examine the complementarity of modularity with domain-adapted representations, we fine-tuned **SBERT** on each dataset before constructing the document graph. Although LR(F) and Linear Probing(F) benefit from fine-tuning, **ModTGCN(F)** achieves the highest micro-F1 on most datasets, with substantial improvements over its pre-trained counterpart on MR (+6.6) and Ohsumed (+5.5). Moreover, despite its simpler architecture, ModTGCN matches TensorGCN’s performance on R8 (97.5 vs. 98.0) and comes within 0.5 points on R52. While overall improvements are consistent, the magnitude of gains varies across datasets. We therefore analyze the structural conditions under which modularity is most beneficial. Compared to embedding-only baselines (LR(P) and Linear Probing(P)), ModTGCN achieves slightly lower performance on simpler, high-homophily datasets (MR -2.05 , R8 -0.70 , R52 -0.72 ; homophily 0.70, 0.50, 0.38), where pretrained representations are already near linearly separable, leaving limited room for structural refinement. In contrast, modularity yields substantial gains on structurally complex, low-homophily datasets (Ohsumed

Table 2: Comparison of LLM baselines with ModTGCN. GPT-3 uses **InstructGPT-3** (**text-davinci-003**).

Model	MR	R8	R52	Ohsumed	20NG
LLM Zero-shot					
GPT-3	88.69	90.19	89.06	–	–
ChatGPT	–	60.10	75.23	39.93	58.70
LLM Few-shot (k)					
GPT-3 (k=16)	89.15	91.58	91.56	–	–
ChatGPT (k=2)	–	72.54	81.68	47.05	58.44
ChatGPT (k=5)	–	82.43	90.13	45.39	–
ModTGCN(P)	81.45	97.55	94.54	71.97	90.60
ModTGCN(F)	88.07	98.70	96.14	77.52	91.14

Table 3: Training time (seconds) and epochs for decoupled vs. original TextGCN. Decoupling reduces per-epoch cost, yielding $2 \times$ to $10 \times$ speedup.

Dataset	Decoupled		Original	
	Time (s)	Epochs	Time (s)	Epochs
MR	10.50	111	11.50	16
R8	36.90	191	87.60	97
R52	64.55	227	257.60	163
Ohsumed	80.62	294	241.10	81
20NG	149.60	243	1450.40	111

+2.10, 20NG +11.54; homophily 0.16, 0.19). This trend indicates that modularity refinement is most beneficial in low-homophily regimes with overlapping semantic boundaries, and less impactful on trivially separable datasets. Overall, these results demonstrate that modularity-based optimization remains effective and complementary under both pre-trained and fine-tuned embedding settings.

5.2 Comparison with LLMs

Table 2 compares ModTGCN against zero-shot and few-shot LLM baselines. Reported GPT-3 and ChatGPT scores are taken directly from their respective papers, and missing entries indicate datasets not evaluated in those works. Both **ModTGCN(P)** and **ModTGCN(F)** outperform LLM variants. The advantage is most evident on Ohsumed and 20NG, suggesting that explicit graph-structured supervision can outperform prompting-based inference when labeled graph structure is available.

5.3 Complexity and scalability analysis

The original TextGCN applies two-hop propagation over a single heterogeneous adjacency $A = [0, \mathcal{A}_{dw}; \mathcal{A}_{wd}, \mathcal{A}_{ww}]$, evaluating both document and word branches at each hop. This incurs sparse cost of $O((4E_{dw} + 2E_{ww})H)$ and unnecessary word-node logit computation. In contrast, our decoupled approach follows only the *document-logit* flow (Eq. (6)) reducing sparse operations to $O((2E_{dw} + E_{ww})H)$. This effectively reduces $\approx 50\%$ edge traversals per layer. The decoupled formulation is lossless for document classification, as it is directly derived from the original TextGCN equations. Since loss is computed only on document logits, removing the unused word branch preserves the decision function while eliminating redundant computation.

Empirically (Table 3), the decoupled variant achieves $2\times$ to $10\times$ faster training than the original TextGCN despite using more epochs, indicating much lower per-epoch cost. The improvement is most pronounced on 20NG, where training time drops from 1,450 seconds to 150 seconds, confirming improved scalability.

6 Ablation studies

6.1 Effect of graph-style, edge reweighting, and labeling strategy

Table 4 evaluates three factors affecting modularity training: (i) $\mathcal{G}_{doc}$ construction method; (ii) label-aware edge reweighting; and (iii) supervision source for modularity.

Graph-style (Gr): We evaluate three G_{doc} construction strategies: (i) *TF-IDF inner product* (tf): $S_{ij} = (A_{dw}A_{dw}^\top)_{ij}$, where A_{dw} is the TF-IDF matrix. This approach does not use language models and relies purely on sparse matrix multiplication over lexical features. (ii) *Cosine similarity* (c): $S_{ij} = \cos(e_i, e_j)$, where e_i is the embedding of the i^{th} sentence. (iii) *Gaussian similarity* (g): $S_{ij} = \exp\left(-\frac{\|e_i - e_j\|^2}{2\sigma^2}\right)$; see Section 3.2. The latter two approaches construct the document graph using transformer-based embeddings. *Observations:* Across all datasets, Gaussian (RBF) graphs consistently outperform cosine- and TF-IDF-based similarities, indicating localized kernel affinities better capture semantic neighborhoods. Moreover, among the three encoders in Table 5, **SBERT-all-mpnet-base** achieves the best overall accuracy and is the most consistent across datasets.

Edge reweighting (W): To inject label supervision into the modularity, we amplify same-class edges and attenuate cross-class edges by factors $\alpha > 1$ and $\beta < 1$ over each training document’s top- k neighbors. Edges connecting validation/test nodes keep their original weights to prevent label leakage. This retains semantic similarity while promoting label-consistent communities. *Observation:* Enabling label-aware edge reweighting improves performance by strengthening intra-class similarities, suggesting that modest supervision injected at the similarity level enhances modular community formation.

Labeling strategy (L): We evaluate two supervision choices for the modularity loss: (i) gold labels on training nodes with predicted labels on validation/test nodes (gt), and (ii) predicted labels for *all* nodes (p). *Observations:* Using predictions (p) in place of ground-truth labels (**gt**) often improves performance, particularly under class imbalance. On Ohsumed, for example, Gaussian + Edge-reweighting + Pred achieves 72.0/65.0 vs. 71.5/63.3 with gold training labels. This indicates that prediction-based supervision can regularize modularity optimization and reduce overfitting to a limited set of labeled nodes.

Overall, ModTGCN under {Gaussian, reweighting enabled, predicted labels}({g,T,p}) configuration gives the strongest results. We use the same configuration for all other further experiments. Table 4 also shows that using predicted labels for all nodes performs slightly better than gold-only supervision, indicating robustness to pseudo-label errors. Moreover, modularity increases smoothly over epochs (Figure 2a), suggesting stable community refinement rather than error amplification. A warm-up strategy (ModTGCN(F) warmup), delaying modularity until few epochs in Table 1, yields only marginal differences (−0.03 on MR, −0.30 on R8, −0.02 on R52, −1.18 on Ohsumed, −1.23 on 20NG), confirming that early prediction noise does not significantly affect convergence.

Table 4: Micro/Macro F1 performance (mean $\pm$ std) under different configurations. Graph style: Gr $\in$ (c,tf,g);(c=Cosine, tf=TF-IDF, g=Gaussian), Edge reweighing: W $\in$ (T, F); T - if weighted adjustment is applied, else F, and Labeling strategy: L $\in$ (gt,p); gt & p denote ground-truth and predicted labels for train-set respectively.

Gr	W	L	MR	R8	R52	Obs	20NG
c	T	gt	81.2 $\pm$ 0.1 / 81.2 $\pm$ 0.1	97.4 $\pm$ 0.1 / 92.2 $\pm$ 0.4	94.1 $\pm$ 0.2 / 67.0 $\pm$ 1.5	71.5 $\pm$ 0.3 / 63.3 $\pm$ 0.5	91.1 $\pm$ 0.1 / 90.5 $\pm$ 0.1
c	T	p	81.4 $\pm$ 0.2 / 81.4 $\pm$ 0.2	97.4 $\pm$ 0.2 / 92.2 $\pm$ 0.3	94.3 $\pm$ 0.2 / 69.1 $\pm$ 2.4	71.7 $\pm$ 0.2 / 65.0 $\pm$ 0.4	90.8 $\pm$ 0.2 / 90.1 $\pm$ 0.2
c	F	gt	81.6 $\pm$ 0.1 / 81.6 $\pm$ 0.1	97.1 $\pm$ 0.1 / 92.0 $\pm$ 0.5	93.7 $\pm$ 0.2 / 69.0 $\pm$ 2.0	68.9 $\pm$ 0.3 / 61.9 $\pm$ 0.7	88.2 $\pm$ 0.1 / 87.3 $\pm$ 0.2
c	F	p	80.8 $\pm$ 0.1 / 80.8 $\pm$ 0.1	96.8 $\pm$ 0.1 / 90.5 $\pm$ 2.3	93.9 $\pm$ 0.1 / 67.6 $\pm$ 1.2	68.2 $\pm$ 0.9 / 60.8 $\pm$ 2.1	88.1 $\pm$ 0.2 / 87.3 $\pm$ 0.2
tf	T	gt	76.4 $\pm$ 0.3 / 76.4 $\pm$ 0.3	95.8 $\pm$ 0.3 / 86.6 $\pm$ 1.9	92.9 $\pm$ 0.1 / 66.6 $\pm$ 2.4	68.3 $\pm$ 0.3 / 62.6 $\pm$ 0.2	86.9 $\pm$ 0.1 / 86.2 $\pm$ 0.1
tf	T	p	76.6 $\pm$ 0.2 / 76.6 $\pm$ 0.2	96.3 $\pm$ 0.2 / 88.5 $\pm$ 2.2	92.8 $\pm$ 0.1 / 66.9 $\pm$ 2.2	68.2 $\pm$ 0.3 / 62.5 $\pm$ 0.4	86.7 $\pm$ 0.2 / 86.2 $\pm$ 0.2
tf	F	gt	77.0 $\pm$ 0.1 / 77.0 $\pm$ 0.1	97.0 $\pm$ 0.2 / 91.7 $\pm$ 1.7	93.5 $\pm$ 0.3 / 66.8 $\pm$ 3.9	68.1 $\pm$ 0.1 / 62.4 $\pm$ 0.3	86.5 $\pm$ 0.2 / 86.0 $\pm$ 0.3
tf	F	p	76.8 $\pm$ 0.1 / 76.8 $\pm$ 0.1	97.0 $\pm$ 0.2 / 90.8 $\pm$ 1.5	93.9 $\pm$ 0.1 / 69.4 $\pm$ 1.8	67.9 $\pm$ 0.6 / 61.3 $\pm$ 1.0	86.3 $\pm$ 0.3 / 86.0 $\pm$ 0.2
g	T	gt	81.6 $\pm$ 0.0 / 81.6 $\pm$ 0.0	97.6 $\pm$ 0.3 / 92.7 $\pm$ 1.2	94.5 $\pm$ 0.2 / 67.6 $\pm$ 1.9	71.5 $\pm$ 0.3 / 63.3 $\pm$ 0.7	90.7 $\pm$ 1.0 / 90.0 $\pm$ 0.9
g	T	p	81.4 $\pm$ 0.1 / 81.4 $\pm$ 0.1	97.5 $\pm$ 0.1 / 92.3 $\pm$ 0.4	94.5 $\pm$ 0.1 / 69.9 $\pm$ 3.0	72.0 $\pm$ 0.1 / 65.0 $\pm$ 0.4	90.6 $\pm$ 0.1 / 89.9 $\pm$ 0.1
g	F	gt	82.4 $\pm$ 0.1 / 82.4 $\pm$ 0.1	97.0 $\pm$ 0.2 / 90.2 $\pm$ 1.8	93.9 $\pm$ 0.2 / 67.7 $\pm$ 2.0	70.7 $\pm$ 0.2 / 63.9 $\pm$ 0.5	90.0 $\pm$ 0.1 / 89.1 $\pm$ 0.1
g	F	p	81.1 $\pm$ 0.1 / 81.1 $\pm$ 0.1	96.7 $\pm$ 0.3 / 90.0 $\pm$ 1.6	93.8 $\pm$ 0.2 / 67.2 $\pm$ 1.6	69.2 $\pm$ 0.3 / 62.9 $\pm$ 0.3	90.0 $\pm$ 0.1 / 89.2 $\pm$ 0.1

Table 5: Performance of ModTGCN under {g,T,p} configuration across three pre-trained transformer embeddings.

Dataset	MR	R8	R52	Obs	20NG
SBERT-all-mpnet-base	81.45$\pm$0.05	97.55$\pm$0.06	94.54$\pm$0.11	71.97$\pm$0.15	90.60$\pm$0.10
BERT-base-uncased	76.89 $\pm$ 0.41	96.76 $\pm$ 0.46	93.72 $\pm$ 0.18	67.72 $\pm$ 0.12	86.85 $\pm$ 0.12
RoBERTa-large	77.12 $\pm$ 0.52	96.99 $\pm$ 0.21	93.90 $\pm$ 0.15	68.10 $\pm$ 0.21	86.93 $\pm$ 0.09

6.2 Hyperparameter sensitivity analysis

Table 6 analyzes sensitivity to key parameters modularity resolution γ , loss weight λ , reweighting factors α, β , and neighborhood size k on ModTGCN. The loss weight λ performs best at intermediate range $\{0.25 - 0.5\}$, indicating that classification and modularity objectives should be balanced. Similarly, moderate value $\{\sim 3.0\}$ for resolution parameter γ yield optimal values, indicating that neither overly coarse nor overly fine communities are optimal. Edge reweighting factors α, β and neighborhood size k also show clear optima: smaller α, β $\{\alpha \sim 1.2, \beta \sim 0.4\}$ with a moderate $k \{\sim 10\}$ produce the strongest results. These results indicate that the benefits of modularity optimization are consistent and robust across reasonable parameter choices.

6.3 Optimization dynamics of CE and Modularity

Figure 2a illustrates that cross-entropy decreases rapidly in early epochs, establishing initial decision boundaries. As predictions stabilize, modularity increases steadily, reinforcing community-level alignment. The smooth convergence of total loss suggests complementary optimization rather than objective conflict. Thus, cross-entropy shapes local classification boundaries, while modularity refines global embedding geometry. This behavior is further reflected in silhouette scores (Figure 2b), which rise and stabilize over epochs, indicating improved intra-class cohesion and inter-class separation. *Additional results, plots, dataset details, and detailed derivation of decoupling approach are available on the website[1]*

Table 6: Ablation studies for key hyperparameters γ , λ , α , β , and k . Micro-F1 is reported in (%).

γ	MR	R8	Ohsumed	R52	20NG
0.1	75.7	97.1	72.6	94.4	87.8
0.5	80.0	97.4	69.5	94.4	87.8
1.0	80.6	97.5	70.7	94.5	88.6
3.0	81.6	97.3	71.9	94.4	89.9
5.0	81.4	97.3	71.9	94.0	90.7

λ	MR	R8	Ohsumed	R52	20NG
0.0	68.4	5.5	16.5	0.7	4.6
0.25	81.3	93.7	67.6	94.5	90.6
0.5	81.3	96.8	71.3	94.1	90.4
0.75	79.4	97.4	72.8	93.8	89.9
1.0	76.5	96.8	66.5	93.4	86.0

k	MR	R8	Ohsumed	R52	20NG
4	79.52	97.26	72.57	94.63	90.32
6	79.38	97.30	72.40	94.47	90.28
8	81.04	97.30	72.52	94.55	90.45
10	81.04	97.44	72.42	94.61	90.04
12	80.53	97.44	72.37	94.59	89.95

β	MR	R8	Ohsumed	R52	20NG
0.3	80.5	97.21	72.59	94.7	90.72
0.4	81.29	97.21	72.57	94.7	90.72
0.5	81.23	97.21	72.57	94.7	90.63
0.6	81.20	97.21	72.57	94.7	89.87
0.7	81.26	97.17	72.57	94.7	89.87

α	MR	R8	Ohsumed	R52	20NG
1.0	80.84	97.30	72.57	94.31	90.59
1.2	81.06	97.39	72.54	94.51	90.17
1.4	81.06	97.39	72.57	94.70	90.04
1.6	81.26	97.21	72.59	94.74	89.86
1.8	80.84	97.17	72.57	94.70	89.88

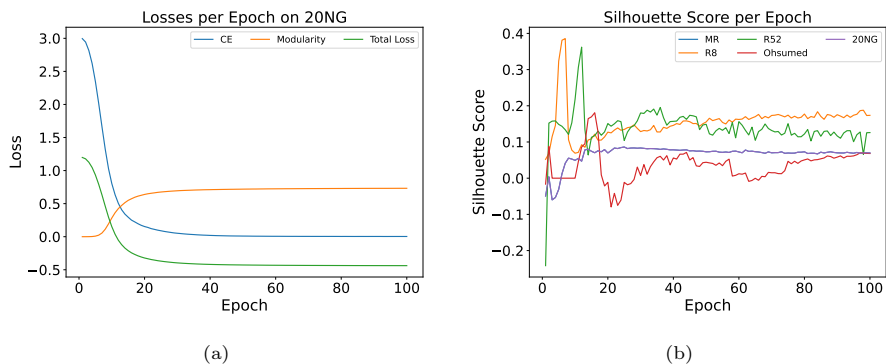


Fig. 2: (a) Cross-entropy, modularity, and total loss across epochs on 20NG. (b) Silhouette score across epochs, showing improved cluster separation during training.

7 Conclusion

We introduced a modularity-aware GNN framework that complements the local neighborhood aggregation with a global, community-aware objective for text classification. By integrating modularity into training, the model encourages class-coherent document communities and mitigates degree-related biases that can lead to over-smoothing or trivial partitions. Experiments across five benchmark datasets demonstrate consistent improvements over baselines. The best results are obtained using Gaussian similarity with prediction-based supervision, underscoring the value of global similarity modeling and hybrid label supervision. Overall, modularity optimization provides a lightweight yet effective complement to GCN-based text classification. Beyond classification, this framework suggests broader opportunities for incorporating community-aware objectives into graph learning tasks where mesoscopic structure plays a central role, including topic modeling, document clustering, and knowledge graph analysis.

References

1. Modtgcg website, <https://sites.google.com/pilani.bits-pilani.ac.in/modtgcg>
2. Barthélemy, M., Fortunato, S.: Resolution limit in community detection. *Bulletin of the American Physical Society* (2007)
3. Bugueño, M., de Melo, G.: Connecting the dots: What graph-based text representations work best for text classification using gnns? In: *EMNLP* (2023)
4. Chen, M., Nguyen, T., Szymanski, B.K.: A new metric for quality of network community structure. *CoRR* (2015)
5. Guo, J., Singh, P., Bassler, K.: Resolution limit revisited: community detection using generalized modularity density. *Journal of Physics: Complexity* (2020)
6. Han, S.C., Yuan, Z., Wang, K., Long, S., Poon, J.: Understanding graph convolutional networks for text classification (2022)
7. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. *CoRR* (2016)
8. Li, W., Aletras, N.: Improving graph-based text representations with character and word level n-grams. In: *Association for Computational Linguistics* (2022)
9. Lin, Y., Meng, Y., Sun, X., et.al.: Bertgcg: Transductive text classification by combining GCN and BERT. *CoRR* (2021)
10. Liu, X., You, X., Zhang, X., Wu, J., Lv, P.: Tensor graph convolutional networks for text classification (2020)
11. Lu, Z., Du, P., Nie, J.: VGCN-BERT: augmenting BERT with graph embedding for text classification. *CoRR* (2020)
12. Murata, T., Afzal, N.: Modularity Optimization as a Training Criterion for Graph Neural Networks. Springer International Publishing (2018)
13. Newman, M.E.J.: Modularity and community structure in networks. *Proceedings of the National Academy of Sciences* (2006)
14. Newman, M.E.J.: Community detection in networks: Modularity optimization and maximum likelihood are equivalent. *CoRR* (2016)
15. Newman, M.E.J., Girvan, M.: Finding and evaluating community structure in networks. *Phys. Rev. E* (2004)
16. Pisarevskaya, D., Zubiaga, A.: Zero-shot and few-shot learning with instruction-following llms for claim matching in automated fact-checking. In: *COLING* (2025)
17. Qiu, C., Huang, Z., Xu, W., Li, H.: VGAER: graph neural network reconstruction based community detection. *CoRR* (2022)
18. Ragesh, R., Sellamanickam, S., Iyer, e.: Hetegcn: Heterogeneous graph convolutional networks for text classification. *WSDM* (2021)
19. Reichardt, J., Bornholdt, S.: Statistical mechanics of community detection. *Phys. Rev. E* (2006)
20. Reimers, N., Gurevych, I.: Sentence-bert: Sentence embeddings using siamese bert-networks. *CoRR* (2019)
21. Salha-Galvan, G., Lutzeyer, J.F., et.al.: Modularity-aware graph autoencoders for joint community detection and link prediction. *CoRR* (2022)
22. Tsitsulin, A., Palowitch, J., Perozzi, B., Müller, E.: Graph clustering with graph neural networks. *CoRR* (2020)
23. Vajjala, S., Shimangaud, S.: Text classification in the llm era – where do we stand? (2025)
24. Yao, L., Mao, C., Luo, Y.: Graph convolutional networks for text classification. *CoRR* (2018)
25. Zhou, D., Bousquet, O., Lal, T.N., et.al.: Learning with local and global consistency. *NIPS’03* (2003)